CSCI1140 Tutorial 5

Play with Memory

Hoover H.F. Yin

Department of Information Engineering
The Chinese University of Hong Kong

October 16, 2015

Poking game!

Contact

- YIN Ho Fai, Hoover
- Office: Institute of Network Coding (INC), 7/F, SHB
- Email: yhf015 at ie dot cuhk dot edu dot hk

- Your local tutor
- Can speak Cantonese :D

Outline

1. Memory
2. Pointer
3. Array
4. Memory and String Operations

Computer Memory

In the view of hardware:

- Basic unit: bit
- Built by memory cells – each store 1 bit

Usually we need multiple bits to represent something...

So in the view of software:

- Basic unit: byte
- Which byte are we reading/writing? Assign each byte an ID!

Memory Address

Every byte in the memory is assigned a number, which we called memory address

- Variables are stored in memory
- Which memory addresses a variable occupied?

address.c (partial)

```
int a = 10;
printf("%p\n", &a);
```

- %p stands for pointer
  - Simple speaking, we are printing the input in a specific format
- The operator & is used to get the address of a variable

Memory Address

The output may change every time you run it...

- It is some kind of security measure...
- You may know the reason if you study CSCI3150/CENG3150/IERG4130/ENGG5105

Here is an example output

Output

```
0x7fff0911996c
```

Memory Address

Output

```
0x7fff0911996c
```

Well, how to read it?

- `0x` means it is a hexadecimal number
- The number `7fff0911996c` is the starting address of the variable

In most modern computers, an `int` occupies 4 bytes

- So the `int` variable `a` in the code occupies the address from `7fff0911996c` to `7fff0911996f`

scanf

Now can you tell the differences between

- `scanf("%d", &a);`
- `scanf("%d", a);`

Suppose `a` is an `int`, it is non-sense to tell `scanf` the current value of `a`

- `scanf` is going to overwrite the variable...
- You should tell `scanf` where the does variable locate!
- In this case, we get the address of `a` by `&a`

If `a` contains a memory address, the second case means

- `scanf` will write the answer in the variable located at `a`

scanf

Here is an example:

```
int a;
int *b = &a;
scanf("%d", &a);
scanf("%d", b);
```

If you can't read the second line, please first review the lecture notes (or the next section in this tutorial)

In fact, it's one of the ways to 'return' multiple values

- A function can only return one value
- If we give the addresses of variables, the function can write the output directly

scanf

- If we pass the address of a variable, it is called call by reference
- If we pass the value of a variable, it is called call by value

Usually a function allows the caller not to receive the 'return' value

- Usually pass a `NULL` means the caller doesn't want to receive the output
- You need to implement it by yourself
  - Don't dereference a `NULL`

Another way to return multiple values?

- Return a `struct`, which can contain multiple members

Pointer in C

The star can be put left or right

```
int *a;
int* b;
int* c, d; //d is not a pointer
```

The data type of `a`, `b`, `c` is `int*`

- The `*` follows the variable name, indicates it's a pointer
- `d` is not a pointer because the line reads as `int(* c), d;`

`int *a` means:

- `int *a` : a is a pointer
- `int *a` : which points to an `int`

What is pointer?

Pointer is...

- A variable which store a memory address
- A memory address is just a number

What are the differences between

- `int*`
- `char*`
- `void*`

Memory address is just a number...

- There is no concept of 'data type' for an address number
  - You can freely do type casting between pointer types
- If I dereference it... '0010101011010110'... How to read it?
  - This is why we have different pointer types

What is pointer?

The differences between different pointer types are

- The data type if you dereference it
- The pointer arithmetic
  - Addition and subtraction of pointers

+1 to a pointer means add the size of the target type to the address

- +1 to `int*` means +4 to the address (suppose `sizeof(int)` is 4)
- +1 to `char*` means +1 to the address (`sizeof(char)` is 1)

Subtraction is similar, by changing + to −

Oh wait, how about `void*`?

void*

The property of `void*`

- You can't dereference it!
- In terms of C standard, no pointer arithmetic on `void*` is allowed
  - Meanwhile, most (not all) compilers allow it
  - If it's allowed, +1 to `void*` means +1 to the address
- It acts as an universal pointer type
  - It is a pointer but we don't know what type it is pointing
  - Or... we just don't care...
- Remember, you can freely type cast between pointer types
  - Cast it to another type before you dereference it

void*

Historical Notes

There was no `void*` in the very old day. People used `char*` for those type-agnostic pointers. After the introduction of `void*`, there was no pointer arithmetic allowed on it. Most compilers extended and allowed the arithmetic. Basically it's just like a `char*` after extension. It can also prevent you from dereference it wrongly in some complicated statements. You can disable the extension from `gcc` by adding `-pedantic` as an argument.

Pointer Arithmetic

Suppose we allow arithmetic on `void*`

pointer.c (partial)

```
int a;
int *foo = &a;
char *bar = &a; //may have warnings
char *bar2 = (char*)&a;
void *haha = &a;
printf("Address of a = %p\n", &a);
printf("foo + 1 = %p\n", foo+1);
printf("bar2 + 1 = %p\n", bar2+1);
printf("haha + 1 = %p\n", haha+1);
```

Pointer Arithmetic

It is okay to assign an `int*` to `char*`

```
char *bar = &a; //may have warnings
```

The compiler warns you because it doesn't sure if you really meant to do it.
To tell the compiler, do a type cast

```
char *bar2 = (char*)&a;
```

Pointer Arithmetic

Here is an example output. The address may change in different runs.

Output

```
Address of a = 0x7fffec2b2324
foo + 1 = 0x7fffec2b2328
bar2 + 1 = 0x7fffec2b2325
haha + 1 = 0x7fffec2b2325
```

As mentioned before

- `foo*` is an `int*` so the address +4
- For `char*` and `void*` (if it's allowed), the address +1

Constant?

const.c (partial)

```
const int a = 10;
int *b = (int*) &a;
*b = 20;
printf("%d\n", a);
```

We can't change a directly as it's a constant. Can we change it through b?

Constant?

Output

20

A local constant variable is not constant at all...

- Global constant variable may be constant, depends on what OS you are using
- CSCI3150/CENG3150 will explain it (advertisement)

`int*` is a pointer to `int`. How about a pointer to `const int`?

- `const int*`
- The pointer of type `const int*` itself is not a constant
- We will discuss how to declare a constant pointer after a few slides...

Point a Pointer

Can we point to a pointer? Pointer of pointer

```
int a;
int *b = &a;
int **c = &b; //NOT &&a
int ***d = &c;
```

How to understand `int **c`?

- You should read it as `int *(*c)`
  - `int **c` : c is a pointer
  - `int **c` : which points to an `int*`

Point a Pointer

How about `int ***d`?

- `int ***d` : d is a pointer
- `int ***d` : which points to a pointer
- `int ***d` : which points to an `int*`

Remember don't type `&&a`

- `&a` means the address of the variable `a`, which is a number
- You want to get the address of a number? It's non-sense

Usually `**` is enough in most situations.

Constant Pointer

```
const int a;
const int *b = &a;
const int * const c = &a;
const int ** const d = &b;
const int * const * const e = &c;
```

The first two are discussed before. How to understand the others?

- `const int * const c` : c is a constant pointer, i.e. c is a constant variable
- `const int * const c` : which points to a `const int`

Constant Pointer

`const int ** const d`?

- `const int ** const d` : d is a constant pointer
- `const int ** const d` : which points to a `const int*`

`const int * const * const e`?

- `const int * const * const e` : e is a constant pointer
- `const int * const * const e` : which points to a constant pointer
- `const int * const * const e` : which points to a `const int`

If it is possible, don't write this kind of code...

Pointer to struct

```
struct foo {
    int *x, y;
};
```

Suppose we declare

- `struct foo a;`

Then a occupied a memory of size `sizeof(struct foo)`

- All the members are inside that allocated memory
- `a.x` means the member `x` inside `a`

Pointer to struct

Suppose we declare

- `struct foo a;`
- `struct foo *b = &a;`

How to dereference `a.x`?

- `*a.x`

How to get `a.y` if we only have `b`?

- `(*b).y`
- `b->y`

Note that the priority of `.` is higher than `*` and `&`
An easy rule to remember

- if the `struct` is a pointer, use `->`
- otherwise, use `.`

Offset in `struct`

What is the address difference between `&a.y` and `&a`?

- The address difference is called the offset of `y` in `struct foo`
- The offset is a constant, no matter which `struct foo` instance you're measuring
- The offset is decided by the compiler to optimize the performance...
  - Word alignment is a speed up, i.e. hold some unused bytes (paddings) between members
  - In most modern computers, the offset (except `char` and `unsigned char`) is a multiple of 4 bytes

Offset in struct

How to get the offset?

- `#include <stddef.h>`
  - Stands for "Standard Definitions"
- Use `offsetof`

For example, how to print the offset of `y` in `struct foo`?

```
printf("%lu\n", offsetof(struct foo, y));
```

Can we disable the paddings?

```
#pragma pack(push,1)
struct bar {
    char a; int b;
};
#pragma pack(pop)
```

union vs struct

What are the differences between

```
union foo {
    char c[4];
    float f;
};
```

```
struct bar {
    char c[4];
    float f;
};
```

- The members in `union` share the same memory space
  - i.e. Same memory space with different names
- Same syntax: use `.` and `->` but replace `struct` by `union`

```
union foo haha;
```

Then, `&haha.c` equals `&haha.f`

* and &

What are the differences between

- `*&a`
- `&*a`

The order of the operators

- `*&a` first get the address of `a`, then dereference it
  - So it is the equivalent to `a`, as long as `a` is a variable
- `&*a` dereference (treat the value of `a` as an address), then get the address of it
  - If `a` is not a pointer, you can't do `*a` (compilation error)
  - Well... you can type cast an `int` into a pointer...

* and &

*a only means the variable located at the address a, but not taking the value out!

```
unsigned long a = 0;
unsigned long b = (unsigned long)&*(unsigned long*)a;
```

It works?

- `(unsigned long)&*(unsigned long*)a` : treat the value of a (i.e. 0) as an address
- `(unsigned long)&*(unsigned long*)a` : get the variable located at address 0
- `(unsigned long)&*(unsigned long*)a` : get the address of the aforementioned variable, i.e. the address is 0
- `(unsigned long)&*(unsigned long*)a` : treat the value 0 as an `unsigned long`

* and &

Can I get the value from the address 0?

- The address 0 is not owned by you
- Runtime error if you take the value in address 0 out, like

```
//runtime error!
int c = *(int*)0UL; //UL means unsigned long
```

- In fact the value of NULL is 0. Verify it by

```
printf("%p\n", NULL);
```

* and &

When will you get the value of a variable? Basically when you really need the value to work on...

- Assign the value to another variable
- Pass by value
- As an array index
- etc...

You may get runtime error if you get the value from a memory location which is not yours

Return a pointer

Never return the address of a local variable!!!

returnpointer.c (partial)

```
int *foo(void)
{
    int a = 10;
    return &a;
}
int main(void)
{
    int *a = foo();
    printf("%d\n", *a);
    printf("%d\n", *a);
    return 0;
}
```

Return a pointer

The two output lines are different! (The second line may change)

Output

```
10
48
```

Why?

- Local variables of a function are stored in the memory
- You don't own that memory after the function returned!
- You can point to it, but it can be replaced by other functions
- In fact, the value in `*a` is replaced by `printf`
- For your interest, search "call stack" on the Internet

`static` local variable

In lecture, we can return the address of a `static` local variable and use it! Why?

- You own the memory for the `static` local variable until your process is terminated

So you can...

- Think it as a global variable, but
- Only accessible directly in that function

static local variable

static.c (partial)

```
void foo(void)
{
    static int count;
    printf("Called %d times\n", ++count);
}
int main(void)
{
    int i;
    for(i = 0; i < 5; i++)
        foo();
    return 0;
}
```

static local variable

The static local variable is initialized by 0. So the output is

Output

```
Called 1 times
Called 2 times
Called 3 times
Called 4 times
Called 5 times
```

Function Pointer

Pointer of a function!!

- In fact, your code is put in the memory for execution...
- So we have the memory address of your code!!!
- Learn more in CSCI3150/CENG3150

What is the data type of a function pointer?

- Replace the function name in the prototype by `(*)`
- For example: `int (*)(int, int)`

How to declare a function pointer?

- The variable name of the pointer is right after the `*`
- For example: `int (*p)(int, int)`

Function Pointer

funcpointer.c (partial)

```
int add(int a, int b)
{
    return a+b;
}
int main(void)
{
    int (*p)(int, int) = add;
    printf("%d\n", p(1,2));
    return 0;
}
```

Function Pointer

Calling `p(1,2)` is equivalent as calling `add(1,2)`

Output

3

Historical Notes

Do we need `&` and `*` for function pointers? It is too complicated for you to call a function by `(*p)(1,2)` so the C standard allowed you to skip them at the very beginning.

Function Pointer

- Can we cast a function pointer to `void*`?
- Yes, but cast it back before you call it

funcpointer2.c (partial)

```
int main(void)
{
    void *p = (void*) add;
    printf("%d\n", ((int (*)(int, int))p)(1,2));
    return 0;
}
```

You will get the same output 3.

Function Table

How about store the function pointers in an array?

- The array is called a function table
- You can then use a `for`-loop to call them one by one!
- The declaration is similar to a function pointer
- For example: `int (*p[2])(int, int)`
- How to read it?
  - `int (*p[2])(int, int)` : `p` is an array with 2 elements
  - `int (*p[2])(int, int)` : where the array elements are function pointers of type `int (*)(int, int)`

Function Table

functable.c (partial)

```
int add(int a, int b)
{
    return a+b;
}
int sub(int a, int b)
{
    return a-b;
}
int main(void)
{
    int (*p[2])(int, int) = {add, sub};
    printf("%d, %d\n", p[0](1,2), p[1](1,2));
    return 0;
}
```

Change data type

Suppose we have a double

- `double d;`

I want to fill an integer `i` to the first 4 bytes of d! Any idea?

- The computer actually doesn't know data types
- The compiler checks you data types
- So fake your compiler :D
- `*(int*)&d = i;`

What is array?

In the view of a process...

Array is a continuous memory space!

So we usually draw in this way...

a[0]	a[1]	a[2]	a[3]	a[4]

What is array?

The following code shows the addresses of each element in an array

array.c (partial)

```
int i;
int a[5];
printf("size of int = %u\n", sizeof(int));
printf("size of a = %u\n", sizeof(a));
for(i = 0; i < 5; i++)
    printf("address of a[%d] = %p\n", i, &a[i]);
```

Now we are going to show that array is a continuous memory space!

What is array?

The addresses may change on different runs, but `a` is an `int` array so the differences are always 4, which is the size of `int`

Output

```
size of int = 4
size of a = 20
address of a[0] = 0x7fffbf27d6e0
address of a[1] = 0x7fffbf27d6e4
address of a[2] = 0x7fffbf27d6e8
address of a[3] = 0x7fffbf27d6ec
address of a[4] = 0x7fffbf27d6f0
```

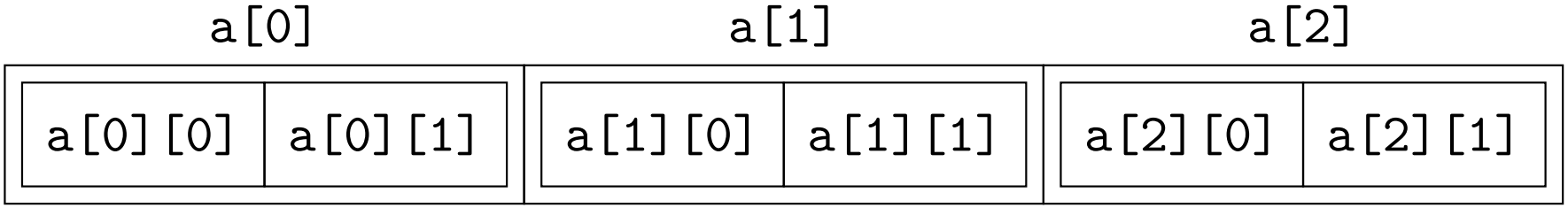
Note: `sizeof(a)` gives the size of the array, which is $5 \times$ `sizeof(int)` $= 20$ in the code

What is 2D-array?

There's no concept of 2D in a series of numbers (memory addresses)... Then what is 2D-array?

- Array of array!

Although we usually draw a table (2D) to express the idea, in fact it's like the following



Because... Array is a continuous memory space!
How about higher dimensional array? Array of array of array of ...

What is 2D-array?

Let's print out the addresses to verify it!

2darray.c (partial)

```
int i, j;
int a[3][2];
printf("size of int = %u\n", sizeof(int));
for(i = 0; i < 3; i++)
    printf("size of a[%d] = %u\n", i, sizeof(a[i]));
printf("size of a = %u\n", sizeof(a));
for(i = 0; i < 3; i++)
    for(j = 0; j < 2; j++)
        printf("address of a[%d][%d] = %p\n", i, j,
                &a[i][j]);
```

What is 2D-array?

The addresses may change on different runs

Output

```
size of int = 4
size of a[0] = 8
size of a[1] = 8
size of a[2] = 8
size of a = 24
address of a[0][0] = 0x7fff5908ae00
address of a[0][1] = 0x7fff5908ae04
address of a[1][0] = 0x7fff5908ae08
address of a[1][1] = 0x7fff5908ae0c
address of a[2][0] = 0x7fff5908ae10
address of a[2][1] = 0x7fff5908ae14
```

Array Index

When you declare an array...

- You allocate a continuous memory space
- The name of the array refer to the address of the first element

So the two lines in the output of the following program are the same!

array0.c (partial)

```
int a[5];
printf("Value of a      = %p\n", a);
printf("Address of a[0] = %p\n", &a[0]);
```

Array Index

We can finally explain why does the array index start from 0

The following are equivalent:

- `a[i]`
- `*(a+i)`

Your computer can only understand `*(a+i)`

- So there's no out-of-bound check!!

If we don't start the index from 0, then we need to do more additions/subtractions to get the address of the element!

- Every operation takes time!

Array Index

Addition is commutative... So the following are equivalent:

- `a[i]`
- `*(a+i)`
- `*(i+a)`
- `i[a]`

It means you can write something like

- `2[a]`
- `"Hello World!"[6]`

It's strange, right?

Array Index

index.c (partial)

```
int i;
int a[3] = {0,1,2};
for(i = 0; i < 3; i++)
    printf("a[i] = %d\n", a[i]);
for(i = 0; i < 3; i++)
    printf("i[a] = %d\n", i[a]);
```

Output

```
a[i] = 0
a[i] = 1
a[i] = 2
```

Output (cont.)

```
i[a] = 0
i[a] = 1
i[a] = 2
```

Negative Index!

If we own the address `a-i`, we can have `a[-i]`!!!

neg.c (partial)

```
int i;
int a[5] = {0,1,2,3,4};
int *b = a+2;
for(i = -2; i <= 2; i++)
    printf("Value of b[%d] = %d\n", i, b[i]);
```

Negative Index!

Output

```
Value of b[-2] = 0
Value of b[-1] = 1
Value of b[0] = 2
Value of b[1] = 3
Value of b[2] = 4
```

Nice!

Address of an array

Guess it!

arrayaddress.c (partial)

```
int a[4];
printf("%p, %p\n", a, &a);
printf("%p, %p\n", a+1, &a+1);
```

- a is the address of the first element, operations in type `int*`
  - But a itself represent the array, so `sizeof(a)` equals 16
- &a is the address of the array, in type `int(*)[4]`
  - `int(*)[4]` : a pointer
  - `int(*)[4]` : which points to an array with 4 elements
  - `int(*)[4]` : every element in the array is of type `int`

Address of an array

The addresses may change on different runs. Note the addresses after addition.

Output

```
0x7fffc9d8e5a0, 0x7fffc9d8e5a0
0x7fffc9d8e5a4, 0x7fffc9d8e5b0
```

- `a+1` adds 4 as expected
- The size of the array is 16, so `&a+1` adds 16

Address of an array

What are these...?

- `int *x[3];`
- `int (*b)[4] = &a;`
- `int (*c[10])[4]; //c[0] = &a;`
- `int (*(*d)[10])[4] = &c;`

Relax, read it part by part, and don't write them in your code∼

`int *x[3]`

- `int *x[3]` : `x` is an array with 3 elements
- `int *x[3]` : every element is of type `int*`

`int (*b)[4]`

- `int (*b)[4]` : `b` is a pointer
- `int (*b)[4]` : which points to an `int` array with 4 elements

Address of an array

`int (*c[10])[4]`

- `int (*c[10])[4]` : c is an array with 10 elements
- `int (*c[10])[4]` : each element is a pointer
- `int (*c[10])[4]` : which points to an array of 4 elements
- `int (*c[10])[4]` :each element is an `int`

`int (*(*d)[10])[4]`

- `int (*(*d)[10])[4]` : d is a pointer
- `int (*(*d)[10])[4]` : points to an array with 10 elements
- `int (*(*d)[10])[4]` : every element is a pointer
- `int (*(*d)[10])[4]` : points to an array of 4 elements
- `int (*(*d)[10])[4]` : every element is an `int`

Golden rule: Read array notation before *

Array as a parameter

How to pass a 2D-array?

funcarray.c (partial)

```
int main(void)
{
    int i, j;
    int a[3][2];// = {{0,1},{2,3},{4,5}};
    for(i = 0; i < 3; i++)
        for(j = 0; j < 2; j++)
            a[i][j] = i*2+j;
    foo(a);
    return 0;
}
```

I want to pass the 2D array a to the function foo

Array as a parameter

funcarray.c (partial)

```
void *foo(int a[][2])
{
    int i, j;
    for(i = 0; i < 3; i++)
        for(j = 0; j < 2; j++)
            printf("value of a[%d][%d] = %d\n", i, j,
                    a[i][j]);
}
```

The function doesn't know how many items in the first dimension...

Array as a parameter

Output

```
value of a[0][0] = 0
value of a[0][1] = 1
value of a[1][0] = 2
value of a[1][1] = 3
value of a[2][0] = 4
value of a[2][1] = 5
```

When you pass an array to a function... You don't need to specify the size of the first dimension! Why?

Array as a parameter

In fact you can also specify it... But the compiler simply skip it...

Suppose we define

- `int a[3][2];`
- `int *b = (int*) a;`

Then...

- Array is a continuous memory space
- `a[i][j]` means `*(b+i*2+j)`
- Computer can only understand `*(b+i*2+j)`
- We don't need to know the size of the first dimension
- But we need to know the size of other dimensions!!

Array as a parameter

2daddress.c (partial)

```
int i, j;
int a[3][2] = {{0,1},{2,3},{4,5}};
int *b = (int*) a;
for(i = 0; i < 3; i++)
    for(j = 0; j < 2; j++)
        printf("Value of *(b+%d*2+%d) = %d\n", i, j,
                *(b+i*2+j));
```

Output

```
Value of *(b+0*2+0) = 0
Value of *(b+0*2+1) = 1
Value of *(b+1*2+0) = 2
```

Output (cont.)

```
Value of *(b+1*2+1) = 3
Value of *(b+2*2+0) = 4
Value of *(b+2*2+1) = 5
```

Memory Allocation

If you want to create an integer array with size `x`, where `x` is a variable...

```
int a[x];
```

This works in GCC but not in VS...

- In fact, constant size array are "embedded" in your code
- The value of `x` is an unknown during compile time
- We need to allocate memory in runtime!

Memory Allocation

Use `malloc`, please `#include <stdlib.h>`

```
int *a = (int*) malloc(x*sizeof(int));
```

- Pass the number of bytes you want to `malloc`
- The allocated memory is yours until you `free` it
  - or your process is terminated
- Return `NULL` if the allocation failed

Free your allocation by

```
free(a);
```

Flexible Array Size in strcut

Let's see this struct

struct.c (partial)

```
struct foo {
    int x;
    int a[]; // or   int a[0];
} *bar;
```

0-sized array?

- Actually sizeof(struct foo) equals sizeof(int)
  - The array a occupies no space!
- You can use bar->a to get the address of the array

Flexible Array Size in strcut

How to use it?

struct.c (partial)

```
bar = (struct foo*)
        malloc(sizeof(struct foo)+5*sizeof(int));
```

You allocated 5 more `int` space for the `struct`

- A `struct` instance is a piece of memory space
- The offset of `a` equals `sizeof(struct foo)`
- Array is a continuous memory space
- So we allocated the space for the array!

As long as we own the memory space, we can use it!

String

What is a string?

- It's a `char` array
- Use the null character '`\0`' to indicate the end of the string
  - Not `NULL`, which is a pointer
  - Although their numerical value are both 0
- String can contain non-printable characters!

String operations?

- Copying, concatenation, comparison...
- Playing with arrays, pieces of memory

String operations are memory operations!

- So all string and memory operations are declared in `string.h`

String Length

How many bytes are used for the strings

- `"Hello!"`
- `""`

Remember to count the null character! Answer:

- 7
- 1

`strlen` can count the length of a string without the null character!

strlen.c (partial)

```
char *a = "Hello"; //string literal
printf("%u\n", strlen(a));
printf("%u\n", strnlen(a, 3));
```

String Length

`strlen` stands for "string length"

Output

```
5
3
```

The 'n' variant `strnlen` will count at most *n* characters (*n* is the second parameter)

- If the string is longer than *n* characters, `strnlen` will return *n*

Basic string operations

Don't try to change a string literal!!

```
char *a = "Hello";
a[0] = 'h'; //WRONG!
```

A literal means, it is a fixed value in your code.

You need to own the memory of the string in order to modify it!

```
char a[6] = {'H','e','l','l','o','\0'};
```

Too lazy to type? Use `strcpy`!

- Stands for "string copy"
- You can copy a string literal into your `char` array

Basic string operations

Create your own string!

strcpy.c (partial)

```
char b[16];
strcpy(b, "hello");
puts(b);
b[0] = 'H';
puts(b);
strcat(b, " World!");
puts(b);
```

Basic string operations

- `puts` stands for "put stream"
  - It will open a newline after printing the string
  - You can also write it as `printf("%s\n", b);`
- `strcat` stands for "string concatenation"
  - It appends a string on another string

So what is the output?

Output

```
hello
Hello
Hello World!
```

Basic string operations

What will happen if you use = to assign a string?

- String is a `char` array
- If the left hand side is a `char*`
  - You copy the address only!
  - They are the same string!!
- If the left hand side is a `char` array
  - You can't assign an address to an array!
  - Compilation Error

Basic string operations

String comparison : `strcmp`

- How to compare two `int a` and `b`?
- Calculate `a-b`
  - If it's < 0, then $a < b$
  - If it's 0, then $a = b$
  - If it's > 0, then $a > b$
- Similar idea on string!

strcmp.c (partial)

```
printf("%d\n", strcmp("Hello", "hello"));
printf("%d\n", strcmp("world", "wor"));
```

Output

```
-1
1
```

Basic string operations

What means a string is smaller than another string?

- Compare their ASCII code of the first character
- If they are not equal, the string with smaller ASCII is the smaller string
- If they are the same, compare the second character and so on

It is the order in the dictionary! (if case insensitive)

- But... `strcmp` is case sensitive
- Recall the ASCII codes...
  - 'A' is 65, or 0x41
  - 'a' is 97, or 0x61
- So small letters are larger than capital letters!

Basic string operations

"Hello" vs "hello"

- 'H' is smaller than 'h', so the output is negative

"world" vs "wor"

- The first three characters "wor" are the same, so consider the fourth one
- 'l' vs '\0', where the ASCII code of null character is 0
- Clearly, the output is positive

The 'n' versions!

- `strncpy, strncmp, strncat, strnlen`
- At most 'n' characters
- Warning! `strncpy` won't write \0 is there's no \0 for the first 'n' bytes!

Basic memory operations

- The string version will return when a \0 is hit
- The memory version won't!
- `memcpy` vs `strncpy`
- `memcmp` vs `strncmp`
- No functions for concatenation and length
  - `memcpy` can do the concatenation effect
  - It doesn't make sense to 'count' the length...
- The `mem` in the names stand for "memory"

More functions?

- `memset`, `memmove`

Remember to `#include <string.h>`

Basic memory operations

`memset`, set the bytes in the memory to a specific value

memset.c (partial)

```
char str[8];
memset(str, 'A', 7);
str[7] = '\0';
puts(str);
```

Output

```
AAAAAAA
```

Usually we use `memset` to initialize a memory space, e.g. fill in 0s

Basic memory operations

What are the differences between `memmove` and `memcpy`?

memmove.c (partial)

```
char *str = "Hello";
char a[6], b[6];
strcpy(a, str);
strcpy(b, str);
memcpy(a+1, a, 3);
memmove(b+1, b, 3);
puts(a);
puts(b);
```

Basic memory operations

Output

```
HHHlo
HHelo
```

- `memmove` is the `memcpy` with overlapped memory space
- `memmove` will copy out the data in a buffer first

Summary

Memory	Memory address is a number
Pointer	Pointer is a variable which stores a memory address
Pointer Arithmetic	Add 1 to a pointer will add the size of the target type to the address
Structure	Use `a.x` to access a member, `a->x` is equivalent to `(*a).x`
Local Variables	The don't own the memory for local variables after the function returned. So don't return addresses of local variable
Static Local Variables	It is a global variable but only the function can access it directly. It is initialized to 0.

Summary

Pointer Type Casting	You can freely cast a pointer type to another, without changing the content
Array	In the view of a process, array is a continuous memory space
2D Array	Array of array, a continuous memory space
Array Index	`a[i]` is equivalent to `*(a+i)`
Memory Allocation	`malloc`, dynamically allocate a memory space in runtime. Remember to `free` it by yourself
String	String is a char array with null character '\0' to indicate the end of the string

Summary

Basic String Operations	`strlen`, `strcpy`, `strcat`, `strcmp`, and their 'n' variants
Memory Operations	Won't return after hitting a null character
Basic Memory Operations	`memcpy`, `memcmp`, `memset`, `memmove`, and remember to `#include <string.h>`